# Strings, Parsing and Recursion

**Topic 7 of 10**

**4 questions | skeleton and writing space only**

**Each question: the prompt, the function or class signature, and ruled lines to write your own solution.**

## 1. Mahjong Winning Hand Detection (14 Tiles)

**MEDIUM | asked 2026-10-09 | recursion, simulation**

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. Input: array of 14 integers representing tile values (clarify suit / range during the interview). A winning hand consists of: One pair (two identical tiles), and Four melds, each being either: A triplet (3 identical tiles), or A run (3 consecutive ranks in the same suit).

```
def is_winning_hand(hand, melds=4):
```

## 2. Snake Case to Camel Case with Repository-Wide Refactor

**MEDIUM | asked 2026-07-14 | string, refactoring**

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions. Implement a conversion from a valid snake_case identifier to camelCase. The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator. Example: foo_bar -> fooBar.

```
def snake_to_camel(name):
```

## 3. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

**MEDIUM | asked 2026-02-01 | string, simulation**

**Examples: text = "Hello world\nFoo bar baz qux" width = 10 Line 1: "Hello" (5 chars; "world" + space would be 11 -> break) Line 2: "world" Line 3: "Foo bar" (7 chars; "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux" Output: 4**

```
def wrap_count(text, width):
```

## 4. Matrix Flower Placement with House Adjacency

**HARD | asked 2026-05-31 | grid, backtracking**

**Examples: H 0 0 0 H 0 0 0 0 0 0 H 0 0 0 0 The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.**

```
def place_flowers(grid, exactly_one=False, find
  _all=False):
```